



NOVA CAPITAL

Plataforma de Coinversión



Autor: Àngel Martínez García

Grado: Desarrollo de Aplicaciones Multiplataforma

Curso: 2025 – 2026

Centro: IES La Vereda

Tutor: Victor Pla Soler

LAVEREDA

ESO
BATXILLERAT
CICLES



INDICE

0. Abstract:.....	2
0.1 Introducción del proyecto (Castellano).....	2
0.2 Introducció del projecte (Valencià).....	3
0.3 Introduction of the project (English)	3
1. Introducción:	3
1.1 Breve descripción del producto:	3
1.2 Módulos que involucran:	4
1.3 Tecnologías utilizadas:.....	8
1.4 Objetivos y Requisitos del proyecto.....	10
1.5 Público objetivo:.....	11
1.6 Estudio previo:	11
2. Timeline:.....	12
3. Base de Datos:	15
3.1 Entidad-Relación:	15
3.2 Documentación del modelo ER.....	17
4. Diseño	20
4.1 Diseño general (análisis)	21
4.2 Diseño detallado (diseño técnico).....	21
5. Implantación.....	26
5.1 En el Servidor	27
5.2 En el Clientes	28
5.3 Seguridad.....	29
5.4 Flujo completo de una petición: invertir en un proyecto	33
6. Recursos	33
6.1 Herramientas Hardware	34
6.2 Herramientas Software	34
6.3 Sistemas Operativos.....	34
6.4 Dependencias y paquetes principales.....	34
6.5 Personal	35
7. Conclusiones	36

7.1 Grado de consecución de objetivos	36
7.2 Problemas encontrados	36
7.3 Mejoras.....	36
8. Anexos y documentos complementarios	37
8.1 Manual de usuario	38
8.2 Instrucciones de instalación.....	39
8.3 Script SQL.....	39
8.4 Colección Postman	40
8.5 Repositorio GitHub	41
8.6 Iniciativa Emprendedora (IPE)	42
9. Referencias / Bibliografía.....	43

0. *Abstract:*

0.1 Introducción del proyecto (Castellano)

El proyecto **Nova Capital** consiste en desarrollar una aplicación de coinversión y educación financiera mediante el uso de una moneda virtual llamada *Aurus*. Los usuarios podrán desempeñar el papel de emprendedores, creando proyectos que requieran inversión, o aportando *Aurus* a los proyectos de otros. El sistema incluirá retos diarios, logros y recompensas para fomentar hábitos de ahorro e inversión. Se implementará un modelo entidad-relación para gestionar usuarios, proyectos, transacciones y progresión gamificada.

0.2 Introducció del projecte (Valencià)

El projecte **Nove Capital** consisteix en desenvolupar una aplicació de coinversió i educació financiera mitjançant l'ús d'una moneda virtual anomenada *Aurus*. Els usuaris podran exercir el paper d'emprededors, creant projectes que requiriscuen inversió, o aportant *Aurus* als projectes d'altres. El sistema inclourà reptes diaris, assoliments i recompenses per a fomentar hàbits d'estalvi i d'inversió. S'implementarà un model entital-reolació per a gestionar usuaris, projectes, transaccions i progressió gamificada.

0.3 Introduction of the project (English)

The **Nova Capital** project aims to develop an application for co-investment and financial education using a virtual currency called *Aurus*. Users can act as entrepreneurs, creating projects that require investment, or contributing *Aurus* to other users' projects. The system will include daily challenges, achievements, and rewards to encourage saving and investments habits. An entity-relationship model will be implemented to manage users, projects, transactions, and gamified progression.

1. *Introducción:*

1.1 Breve descripción del producto:

El proyecto **Nova Capital** se centra en el desarrollo y gestión de una plataforma de coinversión y educación financiera que permite a los usuarios simular procesos reales de inversión de forma segura y gamificada mediante el uso de una moneda virtual llamada *Aurus*.

Nova Capital está diseñada para atraer a un amplio abanico de usuarios, ofreciéndoles la posibilidad de actuar tanto como emprendedores, publicando proyectos que requieren financiación, como inversores, aportando Aurus a las iniciativas de otros. El sistema también incluye retos diarios, logros y recompensas que fomentan los hábitos de ahorro e inversión.

La estructura técnica del proyecto incorpora una base de datos relacional para almacenar información sobre usuarios, proyectos, transacciones y progresión gamificada. Además, se implementa un mecanismo automatizado para calcular los porcentajes de financiación de los proyectos y registrar las contribuciones de los usuarios.

La gestión de Aurus contempla bonificaciones, recompensas por retos e inversiones, todo ello estructurado mediante un modelo de datos robusto con registros de transacciones. El proyecto también incluye el diseño de un **Modelo Entidad-Relación (E/R)** y su conversión a tablas relacionales, garantizando una estructura de datos organizada y eficiente.

1.2 Módulos que involucran:

1. Sistemas Informáticos – (1º Curso) - Marc Crespo

- **RA1: Evalúa sistemas informáticos** → Se aplica al analizar las características del equipo servidor donde se aloja Nova Capital.
- **RA3: Gestiona la información del sistema** → Permite asegurar la integridad y el correcto almacenamiento de los datos del proyecto.
- **RA5: Interconecta sistemas en red** → Aplicado para conectar el servidor, la base de datos y la interfaz del usuario dentro de la misma red.
- **RA6: Opera sistemas en red** → Se asegura el acceso seguro a los recursos del sistema.

2. Bases de Datos – (1º Curso) - Antonio López

- **RA2: Crea bases de datos** → Se utiliza para diseñar la base de datos relacional que gestiona usuarios, proyectos e inversiones.
- **RA3: Consulta la información** → Aplicado en la obtención de datos mediante SQL en las operaciones del sistema.
- **RA5: Desarrolla procedimientos almacenados** → Usado para automatizar cálculos y validaciones internas.
- **RA6: Diseña modelos relacionales** → Implementado en la creación del diagrama entidad-relación de Nova Capital.

3. Programación – (1º Curso) - Xavier Rosillo

- **RA2: Escribe programas sencillos aplicando POO** → Base del desarrollo del backend con clases que representan usuarios, proyectos y transacciones.
- **RA4: Desarrolla programas organizados en clases** → Usado para estructurar el código en Java de forma modular.
- **RA8: Utiliza bases de datos orientadas a objetos** → Aplicado en la conexión del backend con la base de datos.
- **RA9: Gestiona información almacenada** → Garantiza la consistencia e integridad de los datos manipulados por la aplicación.

4. Lenguajes de Marcas – (1º Curso) - Rubén Zarzo

- **RA2: Utiliza lenguajes de marcas para la presentación web** → Aplicado en la definición de los layouts XML de la app Android, que son lenguajes de marcas para la presentación visual de la interfaz.
- **RA6: Gestiona información en formatos de intercambio** → Se utiliza para enviar y recibir datos en formato JSON entre el frontend Android y el backend Spring Boot.

5. Entornos de Desarrollo – (1º Curso) - Leyre Benítez

- **RA2: Evalúa entornos integrados de desarrollo** → Usado para configurar y trabajar en IntelliJ IDEA y Android Studio con GitHub.
- **RA3: Verifica el funcionamiento de programas** → Aplicado en la fase de pruebas de Nova Capital mediante Postman y el emulador Android.
- **RA5: Genera diagramas de clases** → Utilizado para representar la estructura del sistema antes de programar.
- **RA6: Genera diagramas de comportamiento** → Aplicado en la descripción del flujo completo de una petición, que representa el comportamiento del sistema desde la app Android hasta la base de datos.

6. Acceso a Datos – (2º Curso) - Victor Pla

- **RA2: Gestiona información en bases de datos relacionales** → Aplicado al conectar Java con MySQL mediante Spring Data JPA.
- **RA3: Gestiona la persistencia de datos con ORM** → Implementado mediante Hibernate ORM con Spring Data JPA, que mapea automáticamente las entidades Java a las tablas MySQL.
- **RA6: Programa componentes de acceso a datos** → Desarrollado en las clases repositorio que gestionan las consultas SQL a través de JPA.

7. Desarrollo de Interfaces – (2º Curso) - Francisco Hernández

- **RA1: Genera interfaces gráficas de usuario** → Aplicado en la creación del diseño visual de Nova Capital con Android Studio.
- **RA4: Diseña interfaces gráficas aplicando usabilidad** → Usado para crear una interfaz clara, profesional y atractiva siguiendo los principios de Material Design.
- **RA6: Documenta aplicaciones** → Aplicado al crear la memoria técnica del proyecto.

8. Programación Multimedia y Dispositivos Móviles – (2º Curso) - Leyre Benítez

- **RA1: Aplica tecnologías de desarrollo móvil** → Aplicado en el desarrollo de la app Android nativa con Android Studio, utilizando las tecnologías y librerías específicas del entorno móvil como Retrofit2, RecyclerView y Material Design.
- **RA2: Desarrolla aplicaciones para dispositivos móviles** → Permite que Nova Capital sea usable en dispositivos móviles Android mediante una aplicación nativa.

9. Programación de Servicios y Procesos – (2º Curso) - Bernardo Magraner

- **RA3: Programa mecanismos de comunicación en red** → Aplicado en la conexión cliente-servidor mediante peticiones HTTP con Retrofit2.
- **RA4: Desarrolla aplicaciones que ofrecen servicios en red** → Implementado en la API REST de Spring Boot que gestiona usuarios, proyectos e inversiones online.
- **RA5: Protege las aplicaciones y los datos** → Se aplica mediante JWT, JwtAuthFilter y BCrypt para el control de acceso seguro y el almacenamiento protegido de contraseñas.

10. Sistemas de Gestión Empresarial – (2º Curso) - Marc Crespo

- **RA1: Identifica sistemas ERP y CRM** → Aplicado al analizar cómo Nova Capital replica las funciones básicas de un CRM, gestionando el ciclo de vida completo del cliente desde el registro hasta el historial de transacciones.
- **RA2: Implanta sistemas ERP-CRM** → Aplicado en el diseño modular del backend, donde cada servicio gestiona una entidad independiente igual que los módulos de un ERP.
- **RA3: Realiza operaciones de gestión y análisis** → Usado en los endpoints de consulta de la API y en la tabla Transaccion, que registra saldo anterior y posterior en cada movimiento como un sistema de auditoría empresarial.

11. Proyecto de Desarrollo de Aplicaciones – (2º Curso) - Victor Pla

- **RA1: Identifica necesidades del sector** → Aplicado al detectar la necesidad de una plataforma de inversión educativa accesible, analizada en el estudio previo del apartado 1.5.
- **RA2: Diseña el proyecto** → Implementado en el planteamiento, diseño y desarrollo completo de Nova Capital.
- **RA3: Planifica la ejecución** → Aplicado en el cronograma de desarrollo semanal recogido en el apartado 2.
- **RA4: Define procedimientos de seguimiento** → Usado para evaluar el progreso y el cumplimiento de los objetivos del proyecto mediante los informes semanales.

1.3 Tecnologías utilizadas:

Para el desarrollo del proyecto se han seleccionado una serie de tecnologías, las cuales han sido enseñadas y estudiadas en el centro, tanto lenguajes como frameworks.

Frontend

En el frontend se encuentra la experiencia del usuario que va a utilizar la aplicación. Para ello se han utilizado las siguientes tecnologías:

- **Java (Android Studio)** como entorno principal para el desarrollo de la aplicación móvil.
- **XML** para la definición de interfaces gráficas en Android.
- **Java** para la lógica de la interacción con el usuario en la capa de presentación.
- **Material Design** como guía de estilos para asegurar una interfaz moderna y usable.

Backend

El backend se encarga de toda la lógica del servidor, gestión de datos y seguridad de la aplicación, donde se gestionan las llamadas del front y se devuelven los resultados deseados. Las tecnologías utilizadas son las siguientes:

- **Java con Spring Boot** como framework principal para la construcción de la API REST.
- **Postman** para la prueba y validación de las llamadas a la API.
- **Maven/Gradle** como gestor de dependencias y compilación.
- **Routing (Spring)** para definir las rutas de las peticiones HTTP.
- **Middleware (Spring Security básico)** para filtrar y asegurar las peticiones.

Base de datos

Para la gestión de datos del sistema, se ha seleccionado una base de datos relacional que permita integridad y eficiencia:

- **MySQL** como sistema gestor de base de datos principal.
- **SQL (DDL/DML)** para la definición y manipulación de datos.
- **Modelo Entidad-Relación (E/R)** como punto de partida para el diseño estructurado de la base de datos.

Control de versiones

Para gestionar el control de versiones del proyecto se ha utilizado **Git**, lo que permite mantener un historial completo de los cambios realizados, además de servir como repositorio remoto en **GitHub** para colaboración y respaldo del proyecto.

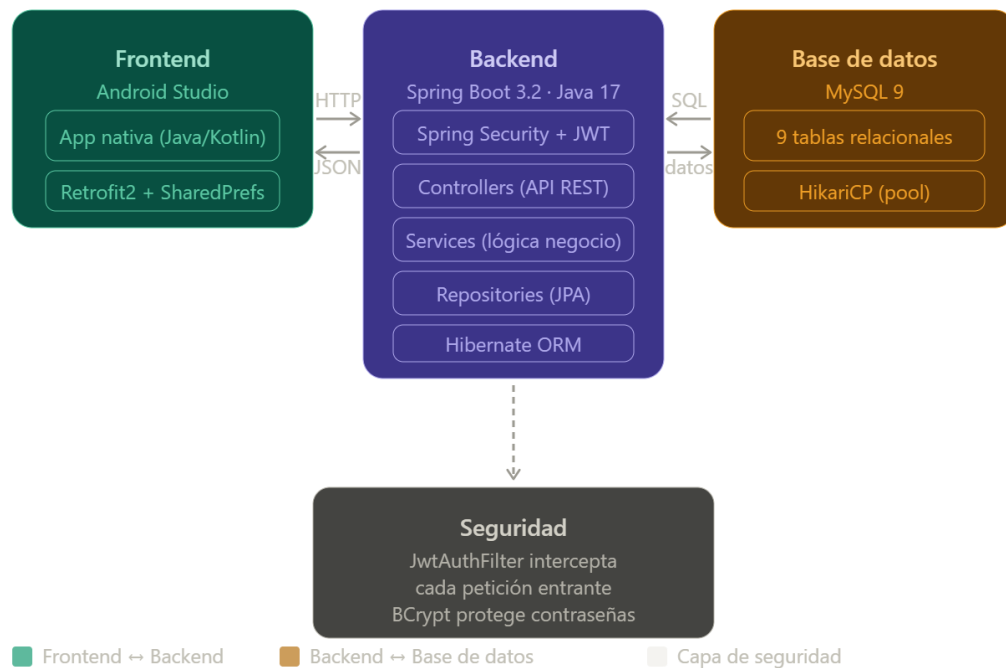


Ilustración 1. Diagrama de arquitectura de Nova Capital.

1.4 Objetivos y Requisitos del proyecto

El objetivo del proyecto **Nova Capital** es crear una aplicación móvil que simule un entorno de **inversión** conjunta, permitiendo que los usuarios aprendan cómo funciona el mundo de la inversión y el ahorro de manera práctica y segura.

La **aplicación** busca **educar** financieramente al usuario mediante una experiencia gamificada, en la que se utilizan monedas virtuales llamadas *Aurus*. Los usuarios pueden crear proyectos, invertir en los de otros, completar retos diarios y ganar recompensas, comprendiendo así conceptos como la rentabilidad, la diversificación y la gestión del riesgo.

Requisitos del proyecto:

- Debe permitir el registro e inicio de sesión de usuarios.
- Cada usuario debe poder actuar como inversor o emprendedor.
- Los emprendedores deben poder crear proyectos con un monto total y una cantidad invertida inicial.
- Los inversores deben poder invertir Aurus en distintos proyectos.
- El sistema debe registrar todas las transacciones e inversiones realizadas.
- Incluir retos y logros diarios para fomentar la participación y el aprendizaje.
- Ofrecer una interfaz atractiva y sencilla basada en los principios de Material Design.

1.5 Público objetivo:

Nova Capital está dirigida a jóvenes y adultos interesados en **aprender** sobre **inversión** y **gestión** financiera sin utilizar dinero real. Incluye estudiantes, emprendedores y personas con interés en la economía o las finanzas personales.

Los emprendedores pueden crear proyectos simulando la búsqueda de socios o financiación, mientras que los inversores pueden aportar monedas virtuales (Aurus) a distintos proyectos y observar su evolución.

La **aplicación** se usa como una herramienta **educativa** y práctica, que fomenta el ahorro, la toma de decisiones y el pensamiento inversor de forma sencilla y segura.

1.6 Estudio previo:

Antes del desarrollo de Nova Capital, se analizaron distintas plataformas relacionadas con la inversión, la educación financiera y el crowdfunding, con el objetivo de identificar qué existe actualmente y qué hueco puede cubrir este proyecto.

• Plataformas de inversión real

Aplicaciones como eToro, Degiro o Interactive Brokers permiten operar en mercados financieros reales. Cabe destacar que determinados brokers si que ofrecen una cuenta demo con dinero virtual, sin embargo, replican mercados reales con activos con acciones o criptomonedas, lo cual, resulta complejo para un usuario sin experiencia. Nova Capital apuesta por un enfoque más sencillo y accesible, con proyectos creados por los propios usuarios en lugar de activos financieros reales.

• Plataformas de crowdfunding

Herramientas como Kickstarter o GoFundMe permiten financiar proyectos entre varios usuarios. Sin embargo, no poseen un enfoque educativo ni sistema de gamificación que fomente la participación continua del usuario.

• Aplicaciones de educación financiera

Aplicaciones como Fintonic Mint ayudan a gestionar gastos personales y a tomar conciencia del dinero, pero no incluyen ningún sistema de inversión ni interacción entre usuarios. Son herramientas pasivas que no simulan la toma de decisiones financieras reales.

• Conclusión del análisis

Tras revisar estas soluciones se detecta un hueco claro: no existe una plataforma que combine la simplicidad de una app educativa, la mecánica de coinversión entre usuarios y un sistema de gamificación que motive el aprendizaje. Nova Capital nace precisamente para cubrir esa necesidad, ofreciendo un entorno seguro donde invertir y aprender sin riesgo económico real.

2. Timeline:

Primera semana

Durante la primera semana estuve analizando los requisitos del proyecto y diseñando el modelo Entidad-Relación de la base de datos. También comencé con los primeros mockups de las pantallas de la app Android.

TAREAS	L	M	X	J	V
Diseño modelo E/R	✓	✓	✓		
Mockups Android			✓	✓	✓

Segunda semana

Durante la segunda semana creé el script SQL completo de la base de datos (DDL y DML) y configuré el proyecto Spring Boot en IntelliJ con las dependencias necesarias (JPA, Security, JWT, MySQL).

TAREAS	L	M	X	J	V
Creación BD MySQL	✓	✓			
Configuración Spring Boot			✓	✓	✓

Tercera semana

Me dediqué a desarrollar la capa de entidades y repositorios JPA del backend, creando las clases Java para Cliente, Proyecto, Inversion, Aurus, Reto, Transaccion, Notificacion y Puntuacion.

TAREAS	L	M	X	J	V
Entidades JPA	✓	✓	✓		
Repositorios Spring Data			✓	✓	✓

Cuarta semana

Desarrollé la capa de seguridad con Spring Security y JWT. Implementé el JwtUtil, JwtAuthFilter y SecurityConfig para proteger los endpoints de la API. También resolví el problema de referencia circular entre beans con @Lazy.

TAREAS	L	M	X	J	V
Spring Security + JWT	✓	✓	✓		
AuthService y AuthController				✓	✓

Quinta semana

Implementé los servicios y controladores REST para todas las funcionalidades: autenticación, proyectos, inversiones, retos, puntuaciones y notificaciones. Creé todos los DTOs para evitar bucles de serialización JSON.

TAREAS	L	M	X	J	V
Services y Controllers	✓	✓	✓	✓	
DTOs y ApiResponse				✓	✓

Sexta semana

Probé todos los endpoints con Postman y corregí errores. Resolví problemas de conexión con MySQL 9 (BadAuthenticationPlugin) actualizando el conector a la versión 9.1.0. También corregí el tipo de dato TINYINT→INT en la tabla Puntuacion.

TAREAS	L	M	X	J	V
Pruebas con Postman	✓	✓			
Corrección errores MySQL			✓	✓	
Registro usuarios de prueba					✓

Séptima semana

Desarrollé el frontend Android en Android Studio. Creé todas las Activities, layouts XML, drawables y el tema oscuro de la app. Configuré Retrofit2 para las llamadas a la API.

TAREAS	L	M	X	J	V
Activities y layouts XML	✓	✓	✓		
Retrofit2 y ApiService				✓	✓

Octava semana

Implementé los adaptadores RecyclerView para proyectos, inversiones y retos. Conecté todas las pantallas con los endpoints del backend y probé el flujo completo login → ver proyectos → invertir desde el emulador Android.

TAREAS	L	M	X	J	V
RecyclerView adapters	✓	✓			
Integración frontend-backend			✓	✓	✓

3. Base de Datos:

3.1 Entidad-Relación:

Imagen del Entidad-Relación:

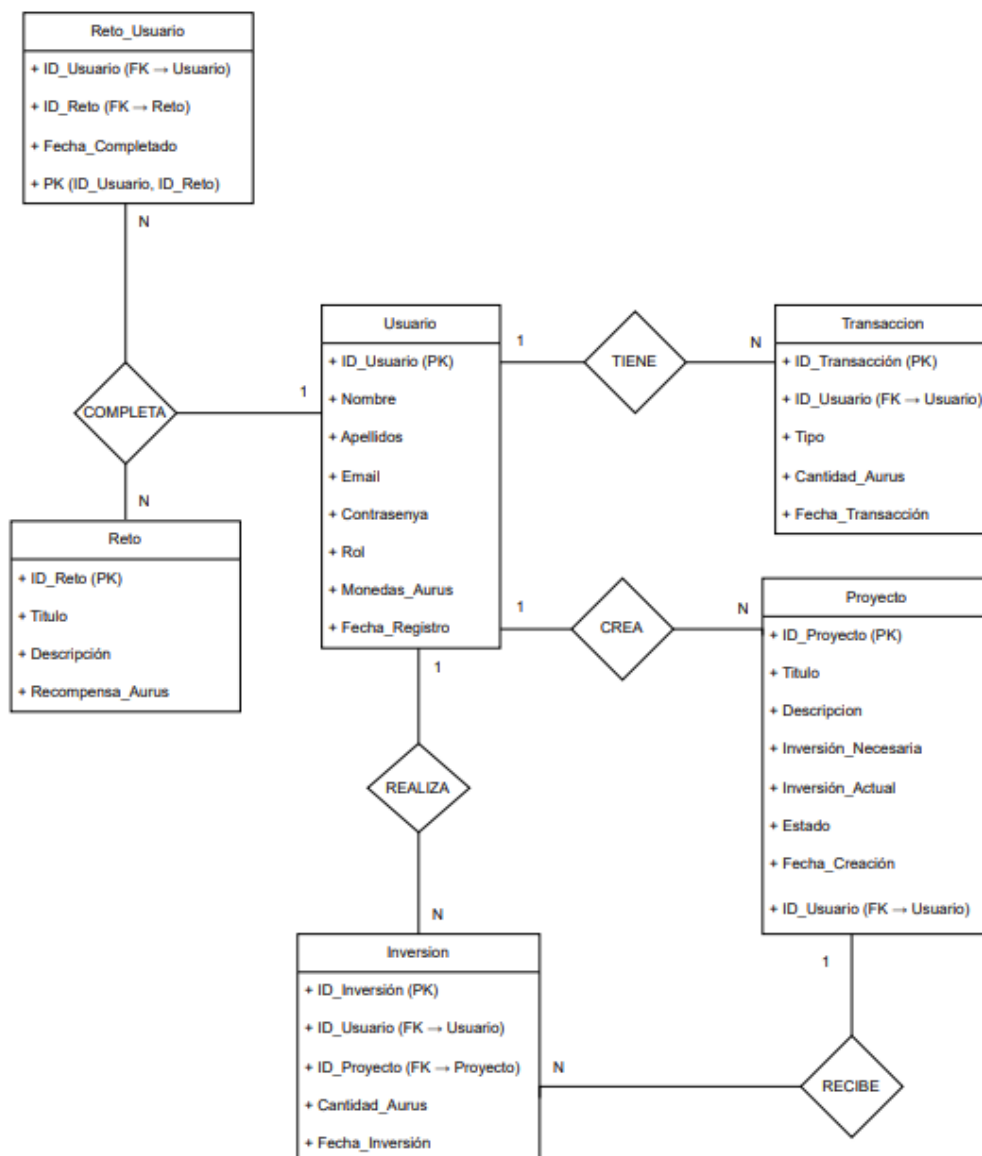


Ilustración 2. Modelo Entidad-Relación de Nova Capital.

```
13 • CREATE TABLE Cliente (  
14     id_cliente      INT AUTO_INCREMENT PRIMARY KEY,  
15     nombre          VARCHAR(100) NOT NULL,  
16     apellidos       VARCHAR(150) NOT NULL,  
17     email           VARCHAR(150) UNIQUE NOT NULL,  
18     contrasena      VARCHAR(255) NOT NULL,  
19     telefono        VARCHAR(20),  
20     activo          BOOLEAN      DEFAULT TRUE,  
21     fecha_registro  DATETIME     DEFAULT CURRENT_TIMESTAMP  
22 );  
23  
24 • CREATE TABLE Proyecto (  
25     id_proyecto      INT AUTO_INCREMENT PRIMARY KEY,  
26     nombre           VARCHAR(150) NOT NULL,  
27     descripcion      TEXT,  
28     categoria        VARCHAR(100),  
29     objetivo_inversion DECIMAL(12,2) NOT NULL CHECK (objetivo_inversion > 0),  
30     cantidad_actual  DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
31     porcentaje       DECIMAL(5,2) GENERATED ALWAYS AS  
32         (ROUND((cantidad_actual / objetivo_inversion) * 100, 2)) STORED,  
33     id_cliente       INT,  
34     estado           ENUM('ACTIVO', 'EN_PROGRESO', 'FINANCIADO', 'CANCELADO') DEFAULT 'ACTIVO',  
35     fecha_creacion   DATETIME     DEFAULT CURRENT_TIMESTAMP,  
36     fecha_actualizacion DATETIME   DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
37     FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)  
38         ON DELETE SET NULL ON UPDATE CASCADE  
39 );
```

Ilustración 3. Script DDL de las tablas Cliente y Proyecto en MySQL.

Explicación del Entidad-Relación:

Este modelo entidad-relación representa un sistema para gestionar usuarios, proyectos e inversiones en la plataforma **Nova Capital**.

La entidad **Usuario** almacena los datos personales y el rol del participante, que puede ser **emprendedor** o **inversor**. Los emprendedores crean **proyectos** con su inversión requerida, cantidad actual y estado.

La entidad **Inversión** une a Usuario y Proyecto en una relación **N:M**, ya que un usuario puede invertir en varios proyectos y un proyecto recibir inversiones de distintos usuarios.

Además, **Transacción** registra los movimientos de la moneda virtual **Aurus**, y **Reto** se vincula con Usuario mediante **Reto_Usuario** para reflejar retos completados y recompensas obtenidas.

3.2 Documentación del modelo ER

A continuación, se describe cada tabla de la base de datos, los campos y relaciones:

1. Cliente

Almacena los datos de los usuarios registrados en la plataforma.

- **id_cliente (PK, INT AUTO_INCREMENT):** Identificador único del cliente.
- **nombre (VARCHAR 100):** Nombre del usuario.
- **apellidos (VARCHAR 150):** Apellidos del usuario.
- **email (VARCHAR 150, UNIQUE):** Correo electrónico, usado para el login.
- **contrasena (VARCHAR 255):** Hash BCrypt de la contraseña.
- **telefono (VARCHAR 20):** Teléfono de contacto opcional.
- **activo (BOOLEAN):** Permite desactivar usuarios sin borrarlos (soft delete).
- **fecha_registro (DATETIME):** Fecha de alta en el sistema.

2. Aurus

Gestiona el saldo virtual de cada cliente. Separado de Cliente para facilitar la auditoría de movimientos.

- **id_aurus (PK, INT):** Identificador.
- **id_cliente (INT, UNIQUE, FK→Cliente):** Cada cliente tiene exactamente un saldo.
- **saldo (DECIMAL 12,2):** Saldo actual en Aurus, arranca en 1000.00.

3. Proyecto

Contiene los proyectos de inversión creados por los usuarios.

- **id_proyecto (PK, INT):** Identificador del proyecto.
- **nombre (VARCHAR 150):** Título del proyecto.
- **descripcion (TEXT):** Descripción detallada.
- **categoria (VARCHAR 100):** Categoría (Tecnología, Agricultura, Educación...).
- **objetivo_inversion (DECIMAL 12,2):** Meta de financiación.
- **cantidad_actual (DECIMAL 12,2):** Cantidad ya invertida.
- **porcentaje (DECIMAL 5,2, GENERATED):** Calculado automáticamente por MySQL.
- **id_cliente (INT, FK→Cliente):** Creador del proyecto.
- **estado (ENUM):** ACTIVO, EN_PROGRESO, FINANCIADO o CANCELADO.
- **fecha_creacion / fecha_actualizacion (DATETIME):** Auditoría de fechas.

4. Inversion

Registra cada inversión realizada por un cliente en un proyecto.

- **id_inversion (PK, INT):** Identificador.
- **id_cliente (INT, FK→Cliente):** Inversor.
- **id_proyecto (INT, FK→Proyecto):** Proyecto en el que se invierte.
- **cantidad (DECIMAL 10,2):** Cantidad invertida en Aurus.
- **fecha_inversion (DATETIME):** Fecha de la operación.

5. Transaccion

Historial completo de todos los movimientos de Aurus de cada usuario.

- **id_transaccion (PK, INT):** Identificador.
- **id_cliente (INT, FK→Cliente):** Usuario implicado.
- **tipo (ENUM):** INVERSION, RECOMPENSA, RETIRO o DEPOSITO.
- **cantidad (DECIMAL 10,2):** Importe del movimiento.
- **saldo_anterior / saldo_posterior (DECIMAL 12,2):** Trazabilidad del saldo.
- **descripcion (VARCHAR 255):** Descripción del movimiento.
- **fecha (DATETIME):** Momento de la transacción.

6. Reto

Define los retos que los usuarios pueden completar para ganar Aurus.

- **id_reto (PK, INT):** Identificador.
- **titulo (VARCHAR 150):** Nombre del reto.
- **descripcion (TEXT):** Condición para completarlo.
- **recompensa (DECIMAL 10,2):** Aurus que se obtienen al completarlo.
- **activo (BOOLEAN):** Permite desactivar retos sin borrarlos.

7. Cliente_Reto

Tabla intermedia N:M que relaciona clientes con retos y registra su estado de completado.

- **id_cliente (INT, FK→Cliente):** Cliente.
- **id_reto (INT, FK→Reto):** Reto.
- **completado (BOOLEAN):** Si el reto ha sido completado.
- **fecha_completado (DATETIME):** Cuándo se completó.
- **PK compuesta (id_cliente, id_reto).**

8. Puntuacion

Almacena las valoraciones de los clientes sobre los proyectos.

- **id_puntuacion (PK, INT):** Identificador.
- **id_cliente (INT, FK→Cliente):** Quien valora.
- **id_proyecto (INT, FK→Proyecto):** Proyecto valorado.
- **valor (INT):** Puntuación del 1 al 5.
- **comentario (TEXT):** Comentario opcional.
- **fecha (DATETIME):** Fecha de la valoración.
- **UNIQUE KEY (id_cliente, id_proyecto):** Un voto por cliente y proyecto.

9. Notificacion

Gestiona los mensajes del sistema hacia los usuarios.

- **id_notificacion (PK, INT):** Identificador.
- **id_cliente (INT, FK→Cliente):** Destinatario.
- **mensaje (VARCHAR 255):** Texto de la notificación.
- **leido (BOOLEAN):** Si el usuario ya la ha leído.
- **fecha (DATETIME):** Fecha de envío.

Relaciones entre tablas

- **Cliente → Aurus: 1 a 1.** Cada cliente tiene exactamente un saldo.
- **Cliente → Proyecto: 1 a N.** Un cliente puede crear varios proyectos.
- **Cliente → Inversion: 1 a N.** Un cliente puede realizar varias inversiones.
- **Proyecto → Inversion: 1 a N.** Un proyecto puede recibir varias inversiones.
- **Cliente ↔ Reto (vía Cliente_Reto): N a M.** Un cliente puede tener varios retos y un reto puede estar asignado a varios clientes.
- **Cliente → Transaccion: 1 a N.** Un cliente puede tener muchas transacciones.
- **Cliente → Notificacion: 1 a N.** Un cliente puede tener varias notificaciones.
- **Cliente → Puntuacion: 1 a N.** Un cliente puede valorar varios proyectos.
- **Proyecto → Puntuacion: 1 a N.** Un proyecto puede recibir varias valoraciones.

4. Diseño

4.1 Diseño general (análisis)

A nivel general, Nova Capital se estructura en tres grandes componentes:

- **Frontend:** Aplicación móvil desarrollada en Android Studio, que permite a los usuarios interactuar con el sistema.
- **Backend:** Servidor desarrollado con Spring Boot, encargado de procesar la lógica del negocio.
- **Base de datos:** MySQL, donde se almacenan usuarios, proyectos, inversiones, retos y transacciones.

El usuario interactúa con la app móvil, que envía peticiones al servidor a través de la API REST. El servidor procesa esas peticiones y consulta o modifica la base de datos, devolviendo la respuesta a la aplicación.

4.2 Diseño detallado (diseño técnico)

En un nivel más concreto, la estructura interna del proyecto se divide en:

• Entidades principales

- Usuario
- Proyecto
- Inversión
- Transacción
- Reto

• Relaciones

- Un usuario puede crear varios proyectos
- Un usuario puede invertir en varios proyectos
- Un proyecto puede recibir inversiones de varios usuarios
- Un usuario puede completar varios retos

• Pantallas principales de la aplicación

- Inicio de sesión / Registro
- Perfil del usuario
- Lista de proyectos
- Detalle del proyecto
- Retos diarios
- Historial de transacciones

Cada una de estas pantallas está conectada al backend mediante métodos HTTP (GET, POST, PUT, DELETE).

El diseño visual de la aplicación se desarrolló directamente sobre el emulador Android, siguiendo los principios de Material Design y usando un tema oscuro con los colores corporativos de Nova Capital: fondo azul oscuro (#0D1B2A), dorado (#D4AF37) para los elementos de acción y blanco para el texto. A continuación, se muestran las pantallas principales de la aplicación funcionando en el emulador.

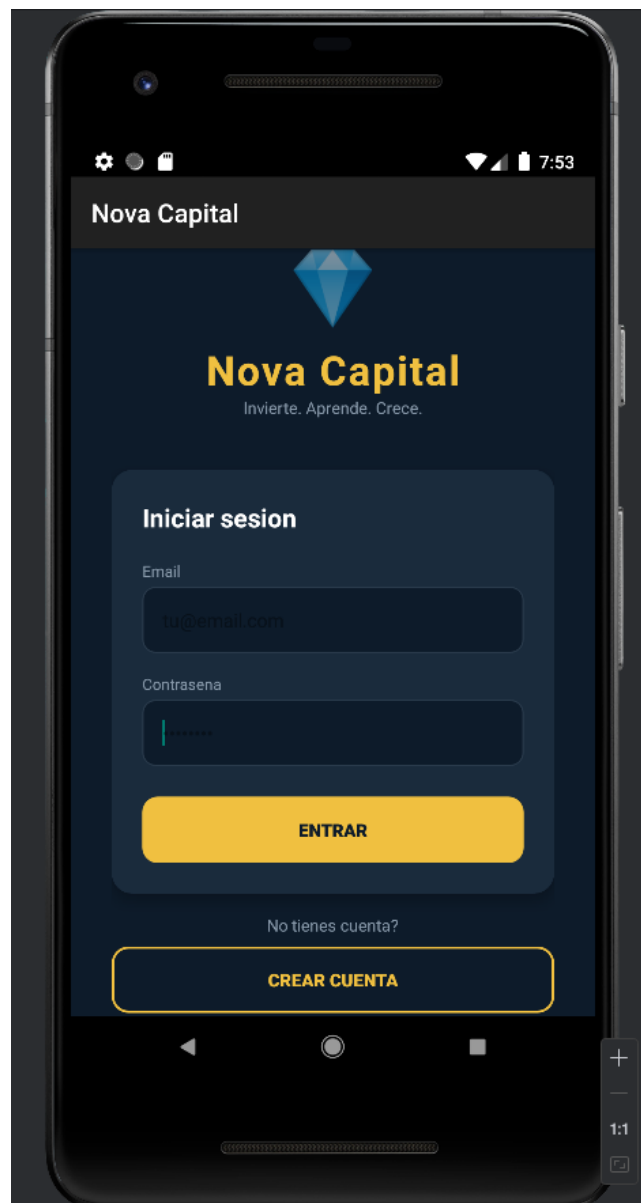


Ilustración 4. Pantalla de inicio de sesión de la aplicación Android.

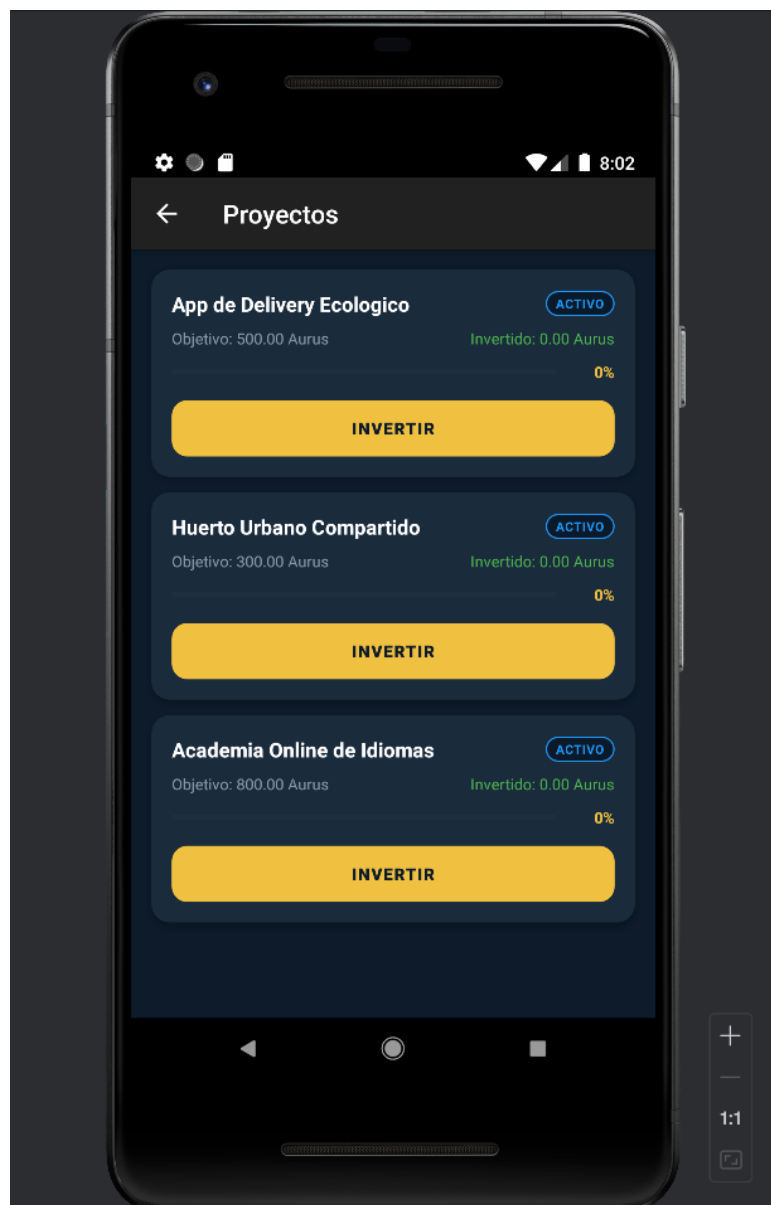


Ilustración 5. Listado de proyectos disponibles en la aplicación Android.

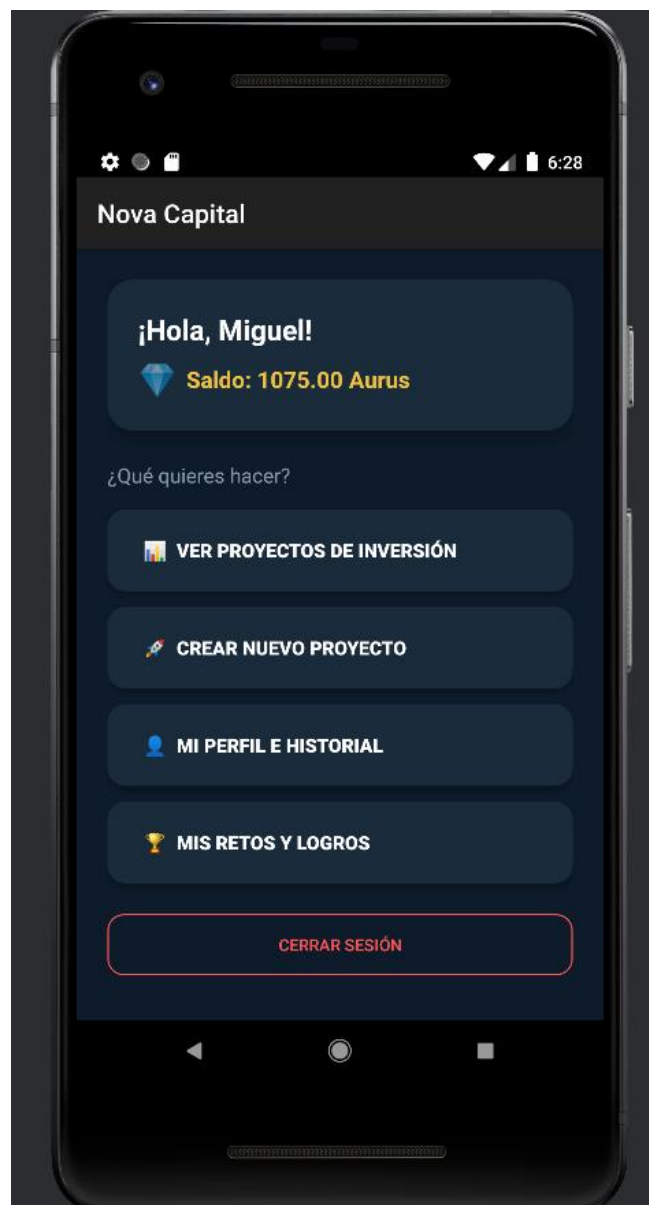


Ilustración 6. Pantalla de perfil del usuario con saldo en Aurus e historial de transacciones.

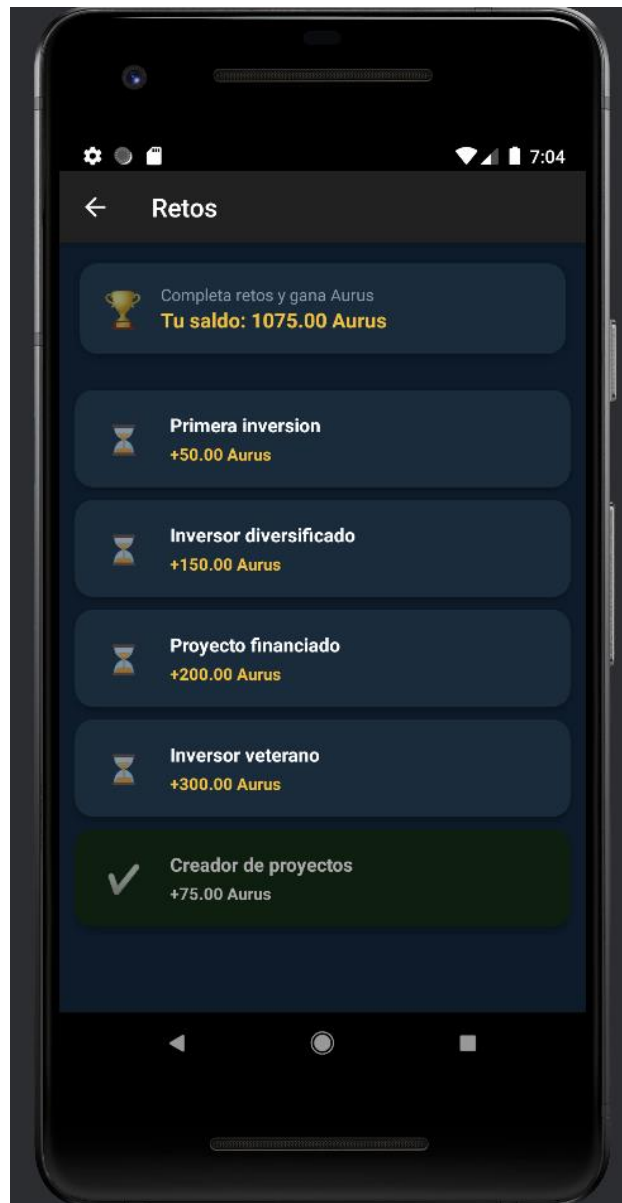


Ilustración 7. Pantalla de retos diarios con recompensas disponibles.

5. Implantación

En este apartado se explican los procesos seguidos para poner en funcionamiento Nova Capital tanto a nivel de servidor como en los clientes.

5.1 En el Servidor

El servidor se basa en Spring Boot 3.2.3 con Java 17. Los pasos seguidos para su puesta en marcha fueron los siguientes.

Primero se instaló el JDK 17 y se configuró IntelliJ IDEA como entorno de desarrollo. Se creó el proyecto incluyendo las dependencias Spring Web, Spring Data JPA, Spring Security, MySQL Driver y la librería JWT (jjwt 0.12.5).

A continuación, se configuró el archivo `application.properties` con la URL de conexión a MySQL, las credenciales de acceso, el puerto del servidor (8080) y el secreto para la firma de tokens JWT.

La base de datos se preparó ejecutando el script SQL en MySQL Workbench, que crea la base de datos Nova Capital con sus 9 tablas y los datos iniciales de los retos. Los usuarios de prueba se registraron a través del endpoint `/api/auth/registro` para que Spring Security los generase los hashes BCrypt correctamente.

Durante el desarrollo se resolvieron dos problemas técnicos relevantes: la incompatibilidad del conector MySQL 8.3.9 con MySQL 9, que se solucionó actualizando el conector a la versión 9.1.0, y una referencia circular entre los beans `JwtAuthFilter` y `SecurityConfig` resuelta añadiendo la anotación `@Lazy` en el constructor de `SecurityConfig`.

Una vez arrancado el servidor con el botón Run de IntelliJ, el mensaje “Started NovaCapitalApplication” en la consola confirma que la API está lista para recibir peticiones en el puerto 8080.

5.2 En el Cliente

El cliente es una aplicación Android nativa desarrollada en Android Studio. Para conectarla con el servidor se configuró Retrofit2 con la URL base <http://10.0.2.2:8080/>, que es la dirección que utiliza el emulador de Android para acceder a localhost del equipo anfitrión.

Cada petición autenticada incluye el token JWT en la cabecera Authorization con el formato Bearer {token}. El token se obtiene en el login y se almacena en SharedPreferences para usarlo en las peticiones posteriores.

Los pasos de puesta en marcha en el cliente son los siguientes: abrir el proyecto en Android Studio, lanzar el emulador, verificar que el backend está corriendo y ejecutar la app. El flujo completo probado fue registro de usuario, inicio de sesión, visualización de proyectos, creación de un proyecto propio, realización de una inversión y consulta de retos y perfil.

```
19      1 usage
20      @POST("api/auth/registro")
21      Call<AuthResponse> registro(@Body RegistroRequest request);
22
23      1 usage
24      @POST("api/auth/login")
25      Call<AuthResponse> login(@Body LoginRequest request);
26
27      3 usages
28      @GET("api/clientes/perfil")
29      Call<ClienteResponse> miPerfil();
30
31      1 usage
32      @GET("api/proyectos")
33      Call<List<ProyectoResponse>> listarProyectos();
34
35      1 usage
36      @GET("api/proyectos/{id}")
37      Call<ProyectoResponse> obtenerProyecto(@Path("id") int id);
38
39      no usages
40      @GET("api/proyectos/mios")
41      Call<List<ProyectoResponse>> misProyectos();
42
43      1 usage
44      @POST("api/proyectos")
45      Call<ProyectoResponse> crearProyecto(@Body ProyectoRequest request);
46
47      1 usage
48      @POST("api/inversiones")
49      Call<InversionResponse> invertir(@Body InversionRequest request);
50
51      1 usage
52      @GET("api/inversiones/mias")
53      Call<List<InversionResponse>> misInversiones();
54
55      1 usage
56      @GET("api/retos/mios")
57      Call<List<ClienteRetoResponse>> misRetos();
```

Ilustración 8. Interfaz ApiService con los endpoints definidos mediante Retrofit2.

5.3 Seguridad

La seguridad de Nova Capital se basa en tres mecanismos que trabajan de forma conjunta: JWT para la autenticación sin estado, JwtAuthFilter para proteger los endpoints de la API, y BCrypt para el almacenamiento seguro de contraseñas.

JWT (JSON Web Token)

Cuando un usuario hace login correctamente, el servidor genera un token JWT firmado digitalmente con una clave secreta. Este token contiene el email del usuario y una fecha de caducidad, y se devuelve al cliente Android, que lo guarda en SharedPreferences. A partir de ese momento, cada petición que envía la app incluye ese token en la cabecera HTTP con el formato *Authorization: Bearer <token>*. El servidor puede verificar el token en cualquier momento sin necesidad de consultar la base de datos sin mantener sesiones activas, lo que lo hace especialmente adecuado para una API REST.

```
34 // Genera un token JWT para el usuario dado.
35 public String generarToken(String email) { 2 usages  ▲ AngelMartinez10
36     return Jwts.builder()
37         .subject(email)
38         .issuedAt(new Date())
39         .expiration(new Date(System.currentTimeMillis() + expiration))
40         .signWith(getKey())
41         .compact();
42 }
43
44 // Comprueba si el token es válido para el usuario dado.
45 @ public boolean esTokenValido(String token, UserDetails userDetails) { 1 usage new *
46     final String email = extraerEmail(token);
47     return email.equals(userDetails.getUsername()) && !estaExpirado(token);
48 }
```

Ilustración 9. Métodos generarToken() y esTokenValido() en JwtUtil.java.

JwtAuthFilter

El JwtAuthFilter es un filtro de seguridad que he implementado para proteger los endpoints de la API. Funciona como un control de acceso que se ejecuta automáticamente en cada petición HTTP antes de que llegue al controlador.

Su funcionamiento es el siguiente: cuando la app Android envía una petición al servidor, el filtro lee la cabecera *Autorization* y extrae el token JWT. A continuación, comprueba que ese token tiene una firma válida y que no ha caducado. Si todo es correcto, obtiene el email del usuario desde el token, carga sus datos desde la base de datos y le da acceso. Si el token no existe o no es válido, la petición es rechazada con un error 403, sin llegar a ejecutar ninguna lógica de negocio.

Tuve que implementarlo porque Spring Security por defecto está pensado para aplicaciones web tradicionales con formularios de login y sesiones. Como Nova Capital es una API REST que autentica mediante tokens JWT, necesitaba decirle a Spring cómo debe validar cada petición entrante, y eso se consigue con este filtro personalizado que actúa de enlace entre el token que llega del cliente y el sistema de seguridad del servidor.

```

37  @Override no usages AngelMartinez10
38  protected void doFilterInternal(HttpServletRequest request,
39                                HttpServletResponse response,
40                                FilterChain filterChain)
41                                throws ServletException, IOException {
42
43      final String authHeader = request.getHeader("Authorization");
44
45      // Si no hay header o no empieza por "Bearer ", dejamos pasar sin autenticar
46      if (authHeader == null || !authHeader.startsWith("Bearer ")) {
47          filterChain.doFilter(request, response);
48          return;
49      }
50
51      // Extraer el token (quitamos "Bearer ")
52      final String token = authHeader.substring(beginIndex: 7);
53      final String email;
54
55      try {
56          email = jwtUtil.extraerEmail(token);
57      } catch (Exception e) {
58          // Token malformado o expirado
59          filterChain.doFilter(request, response);
60          return;
61      }
62
63      // Si hay email y aún no hay autenticación en el contexto
64      if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
65          UserDetails userDetails = userService.loadUserByUsername(email);
66
67          if (jwtUtil.esTokenValido(token, userDetails)) {
68              UsernamePasswordAuthenticationToken authToken =
69                  new UsernamePasswordAuthenticationToken(
70                      userDetails, credentials: null, userDetails.getAuthorities());
71              authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
72              SecurityContextHolder.getContext().setAuthentication(authToken);
73          }
74      }
75
76      filterChain.doFilter(request, response);
77  }

```

Ilustración 10. Método doFilterInternal() en JwtAuthFilter.java.

BCrypt

Las contraseñas de los usuarios nunca se almacenan en texto plano. Al registrarse, la contraseña pasa por el algoritmo BCrypt, que genera un hash irreversible de 60 caracteres. BCrypt incorpora un mecanismo de salt aleatorio, lo que garantiza que dos usuarios con la misma contraseña tendrán hashes completamente diferentes en la base de datos. Al hacer login, Spring Security compara la contraseña introducida con el hash almacenado usando `BCryptPasswordEncoder.matches()`, sin necesidad de revertir el hash. Esto es lo que causaba problema con los usuarios insertados manualmente por SQL: al no pasar por el encoder, sus contraseñas no eran hashes válidos y el login fallaba siempre.

```

56  /**
57   * Registra un nuevo cliente con saldo inicial de 1000 Aurus.
58   * Asigna automáticamente todos los retos disponibles.
59   */
60  @Transactional 1 usage 1 AngelMartinez10
61  @ @ public AuthResponse registrar(RegistroRequest request) {
62      // Comprobar que el email no existe ya
63      if (clienteRepo.existsByEmail(request.getEmail())) {
64          throw new BadRequestException("El email ya está registrado: " + request.getEmail());
65      }
66
67      // Crear cliente con contraseña hasheada
68      Cliente cliente = new Cliente();
69      cliente.setNombre(request.getNombre());
70      cliente.setApellidos(request.getApellidos());
71      cliente.setEmail(request.getEmail());
72      cliente.setContraseña(passwordEncoder.encode(request.getContraseña()));
73      cliente.setTelefono(request.getTelefono());
74      cliente = clienteRepo.save(cliente);
75
76      // Crear saldo Aurus (1000 por defecto)
77      Aurus aurus = new Aurus(cliente);
78      aurusRepo.save(aurus);
79
80      // Asignar todos los retos activos al nuevo cliente
81      List<Reto> retos = retoRepo.findByActivoTrue();
82      for (Reto reto : retos) {
83          ClienteReto cr = new ClienteReto();
84          cr.setId(new ClienteReto.ClienteRetoId(cliente.getIdCliente(), reto.getIdReto()));
85          cr.setCliente(cliente);
86          cr.setReto(reto);
87          cr.setCompletado(false);
88          clienteRetoRepo.save(cr);
89      }
90
91      // Generar token JWT
92      String token = jwtUtil.generarToken(cliente.getEmail());
93      return new AuthResponse(token, cliente.getIdCliente(),
94          cliente.getNombre(), cliente.getEmail(), aurus.getSaldo());
95  }

```

Ilustración 11. Método registrar() con BCryptPasswordEncoder en AuthService.


```

29 public SecurityConfig(ClienteUserDetailsService userDetailsService, new *
30     JwtAuthFilter jwtAuthFilter) {
31     this.userDetailsService = userDetailsService;
32     this.jwtAuthFilter = jwtAuthFilter;
33 }
34
35 // Define qué rutas necesitan token JWT y cuáles son públicas
36 @Bean  * ÁngelMartínez10
37 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
38     http
39         .csrf(csrf -> csrf.disable())
40         .cors(cors -> cors.configurationSource(corsConfigurationSource()))
41         .authorizeHttpRequests(auth -> auth
42             .requestMatchers("/api/auth/**").permitAll()
43             .anyRequest().authenticated()
44         )
45         .sessionManagement(session ->
46             session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
47         .authenticationProvider(authenticationProvider())
48         .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
49
50     return http.build();
51 }
52
53 // Conecta nuestro UserDetailsService y el encriptador de contraseñas con Spring Security
54 @Bean  * ÁngelMartínez10
55 public AuthenticationProvider authenticationProvider() {
56     DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
57     provider.setUserDetailsService(userDetailsService);
58     provider.setPasswordEncoder(passwordEncoder());
59     return provider;
60 }
61
62 @Bean  * ÁngelMartínez10
63 public PasswordEncoder passwordEncoder() {
64     return new BCryptPasswordEncoder();
65 }

```

Ilustración 12. Configuración de rutas y beans de seguridad en SecurityConfig.java.

5.4 Flujo completo de una petición: invertir en un proyecto

A continuación, se describe paso a paso qué ocurre desde que el usuario pulsa el botón “Invertir” en la app Android hasta que el saldo queda actualizado en la base de datos.

1. El usuario introduce la cantidad a invertir y pulsa el botón. La Activity llama al método correspondiente en Retrofit2, que construye la petición *POST /api/inversiones* con el cuerpo en JSON (*id_proyecto* y *cantidad*) y añade automáticamente la cabecera *Authorization: Bearer <token>* con el JWT almacenado en *SharedPreferences*.
2. La petición llega al servidor y es interceptada por el *JwtAuthFilter* antes de llegar al controlador. El filtro valida la firma del token, extrae el email del usuario y lo registra en el contexto de seguridad de Spring. Si el token fuera inválido o estuviese caducado, la petición sería rechazada en este punto con un error 403.
3. La petición pasa al *InversionController*, que recibe el DTO con los datos de la inversión y llama al *InversionService*.
4. El *InversionService* ejecuta la lógica de negocio: comprueba que el proyecto existe y está activo, verifica que el usuario tiene saldo suficiente en su tabla *Aurus*, descuenta la cantidad del saldo del inversor y crea el registro en la tabla *Inversión*, actualiza el campo *cantidad_actual* del proyecto y registra el movimiento en la tabla *Transacción* con los campos *saldo_anterior* y *saldo_posterior* para garantizar la trazabilidad.
5. Todos los cambios se persisten en MySQL a través de los repositorios JPA. Hibernate traduce las operaciones Java a sentencias SQL que se ejecutan en la base de datos.
6. El servidor construye un objeto *ApiResponse* con el código de estado y un mensaje de éxito, y lo devuelve como JSON al cliente Android.
7. Retrofit2 recibe la respuesta en el hilo secundario y la entrega al callback definido en la Activity. Si el código es 200, la interfaz se actualiza mostrando el nuevo saldo y confirmando la inversión realizada.

6. Recursos

6.1 Herramientas Hardware

- Ordenador de desarrollo con Windows 11, procesador intel Core, 16 GB RAM y 512 GB SSD.
- Emulador Android integrado en Android Studio (Pixel 2, Android 8.1).

6.2 Herramientas Software

- IntelliJ IDEA 2024.2 - entorno de desarrollo de backend.
- Android Studio Meerkat – entorno de desarrollo de frontend Android.
- MySQL Server 8.0 en el puerto 3307 – sistema gestor de bases de datos.
- MySQL Workbench - gestión visual de la base de datos.
- Postman – prueba y validación de los endpoints de la API REST.
- Git y GitHub – control de versiones y repositorio remoto.

6.3 Sistemas Operativos

- Windows 11 como sistema operativo del equipo de desarrollo.
- Android 8.1 (API 27) como sistema operativo del cliente móvil del emulador

6.4 Dependencias y paquetes principales

- Spring Boot 3.2.3
- Spring Security 6.2.2
- Hibernate ORM 6.4.4
- Jjwt 0.12.5 (JSON Web Token)
- mysql-connector-j 9.1.0
- Retrofit2 2.9.0
- HikariCP 5.0.1 (pool de conexiones)

6.5 Personal

El proyecto ha sido desarrollado íntegramente por Àngel Martínez García, alumno de 2º de DAM, con la supervisión del tutor Victor Pla Soler.

7. Conclusiones

7.1 Grado de consecución de objetivos

El resultado del proyecto Nova Capital es satisfactorio. Se ha conseguido desarrollar una aplicación fullstack funcional compuesta por un backend Spring Boot con API REST autenticada mediante JWT, una base de datos MySQL relacional con 9 tablas y una aplicación Android nativa conectada al servidor.

Las funcionalidades principales planteadas al inicio del proyecto están implementadas y funcionando: registro e inicio de sesión de usuarios, creación e inversión en proyectos con actualización automática del saldo en Aurus, sistema de retos, historial de transacciones y consulta del perfil del usuario.

7.2 Problemas encontrados

El principal problema técnico fue la incompatibilidad entre MySQL 9 y el conector Java 8.3.0. MySQL 9 eliminó el plugin de autenticación `mysql_native_password`, lo que provocaba `BadAuthenticationPlugin` al intentar arrancar el backend. La solución fue actualizar el conector a la versión 9.1.0, que soporta el nuevo método de autenticación `caching_sha2_password`.

También apareció una referencia circular en el contexto de Spring entre los beans `JwtAuthFilter` y `SecurityConfig`, ya que cada uno dependía del otro en su construcción. Se resolvió extrayendo la lógica de `UserDetailsService` a una clase propia llamada `ClienteUserDetailsService`, eliminando así la dependencia circular entre los dos beans.

Otro problema fue que los usuarios insertados directamente mediante SQL no podían autenticarse, porque los hashes BCrypt introducidos manualmente no eran válidos. La solución definitiva fue registrar siempre los usuarios a través del endpoint `/api/auth/registro`, delegando la generación del hash en `BCryptPasswordEncoder` de Spring Security.

7.3 Mejoras

A pesar de que las funcionalidades principales están implementadas, hay varias mejoras que se podrían incorporar en versiones futuras:

- Sistema de notificaciones push en Android para avisar al usuario cuando su proyecto recibe una inversión.
- Pantalla de ranking de inversores con los usuarios que más Aurus han invertido.
- Gráficas de evolución del saldo en la pantalla de perfil.
- Sistema de recuperación de contraseña por correo electrónico.
- Exportación del historial de transacciones en formato PDF.
- Modo oscuro y claro configurable desde los ajustes de la app.
- Soporte para imágenes en los proyectos para que los emprendedores puedan añadir una foto representativa.

8. Anexos y documentos complementarios

8.1 Manual de usuario

¿Qué es Nova Capital?

Nova Capital es una aplicación móvil de coinversión y educación financiera que permite a los usuarios aprender cómo funciona el mundo de la inversión de forma práctica y segura, utilizando una moneda virtual llamada Aurus.

Funcionamiento General

Para acceder a la aplicación es necesario registrarse con nombre, apellidos, email y contraseña. Al registrarse, cada usuario recibe automáticamente 1000 Aurus como saldo inicial y se le asignan todos los retos disponibles.

El sistema distingue dos roles: emprendedor, que puede crear proyectos que necesitan financiación, e inversor, que puede aportar Aurus a los proyectos de otros usuarios.

Pantallas principales

Inicio de sesión y registro: Al abrir la app aparece la pantalla de login con los campos de email y contraseña. Si es la primera vez, el usuario puede registrarse desde el botón de registro introduciendo sus datos personales.

Lista de proyectos: Muestra todos los proyectos activos de la plataforma con su título, categoría, porcentaje de financiación alcanzado y estado. El usuario puede pulsar sobre cualquier proyecto para ver su detalle.

Detalle del proyecto: Muestra la descripción completa del proyecto, la cantidad necesaria, la cantidad ya invertida y el porcentaje de financiación. Desde aquí el inversor puede introducir una cantidad y pulsar el botón Invertir para aportar sus Aurus.

Crear proyecto: Accesible desde el menú principal. Permite al emprendedor introducir título, descripción, categoría y objetivo de inversión de su proyecto.

Retos diarios: Muestra los retos disponibles con su descripción y la recompensa en Aurus que se obtiene al completarlos. Los retos completados aparecen marcados.

Perfil: Muestra los datos del usuario, su saldo actual en Aurus y el historial de transacciones con cada movimiento registrado, incluyendo la fecha y el tipo de operación.

8.2 Instrucciones de instalación

Instrucciones de instalación

Nova Capital está compuesto por un backend Spring Boot y una app Android nativa. Para ponerlo en funcionamiento es necesario tener arrancado el backend antes de lanzar la app.

Requisitos previos

Para el backend: JDK 17, IntelliJ IDEA y MySQL Server activo en el puerto 3307. Para el cliente: Android Studio con un emulador configurado (Pixel 2, Android 8.1 o superior).

Instalación de backend

Clonar el repositorio desde GitHub y abrir la carpeta del backend en IntelliJ IDEA. Ejecutar el script SQL adjunto en MySQL Workbench para crear la base de datos con sus 9 tablas y los retos iniciales. Verificar que el archivo *application.properties* tiene la URL de conexión correcta, las credenciales de MySQL y el secreto JWT. Pulsar el botón RUN en IntelliJ. Cuando aparezca en la consola el mensaje *Started NovaCapitalApplication* el servidor está listo en el puerto 8080.

Registro de usuarios de prueba

Los usuarios deben registrarse siempre a través del endpoint *POST /api/auth/registro* o desde la pantalla de registro de la app, nunca insertando directamente en la base de datos, ya que Spring Security necesita generar el hash BCrypt de la contraseña correctamente.

Instalación del cliente Android

Abrir el proyecto del frontend en Android Studio. Verificar que en el archivo de configuración de Retrofit la URL base es <http://10.0.2.2:8080/>, que es la dirección que usa el emulador Android para acceder al localhost del equipo. Lanzar el emulador y ejecutar la app. El flujo completo para probar es: registro → login → ver proyectos → invertir → consultar perfil.

8.3 Script SQL

Base de datos Nova Capital

El script DDL/DML completo de la base de datos se encuentra adjunto en el repositorio GitHub del proyecto. Incluye la creación de las 9 tablas con sus restricciones e índices, y la inserción de los 5 retos iniciales del sistema.

8.4 Colección Postman

Durante el desarrollo se utilizó Postman para validar todos los endpoints de la API. Los principales endpoints probados fueron los siguientes:

Método	Endpoint	Autenticación
POST	/api/auth/registro	No requerida
POST	/api/auth/login	No requerida
GET	/api/proyectos	Bearer Token
POST	/api/proyectos	Bearer Token
POST	/api/inversiones	Bearer Token
GET	/api/retos/mis-retos	Bearer Token
GET	/api/clientes/perfil	Bearer Token
GET	/api/notificaciones	Bearer Token

Proyecto Nova Capital

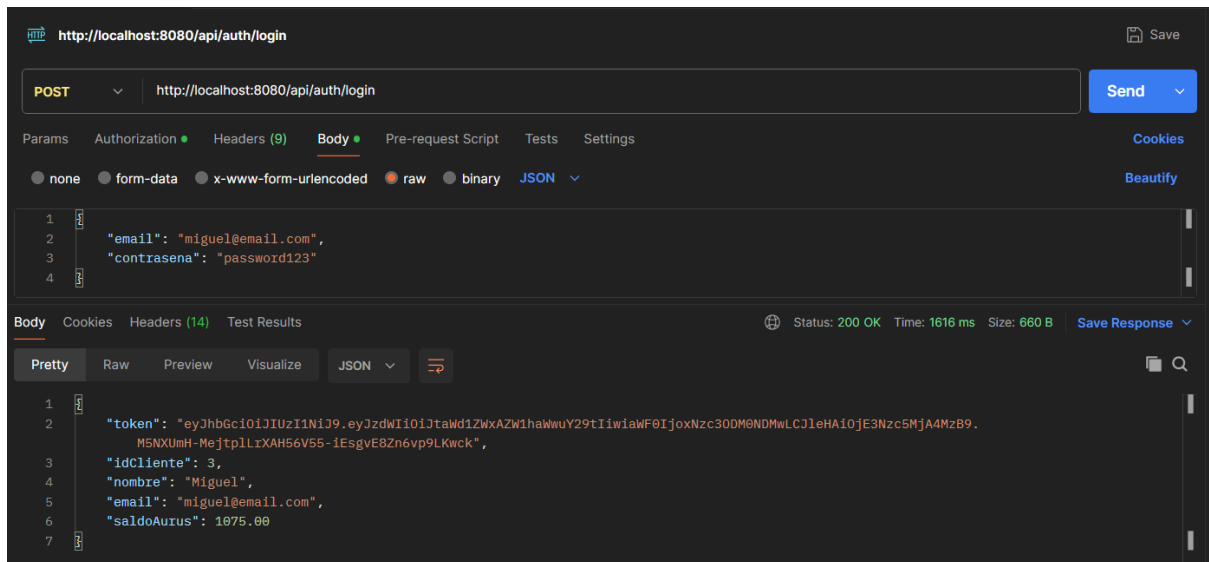


Ilustración 13. Respuesta del endpoint /api/auth/login con el token JWT generado.

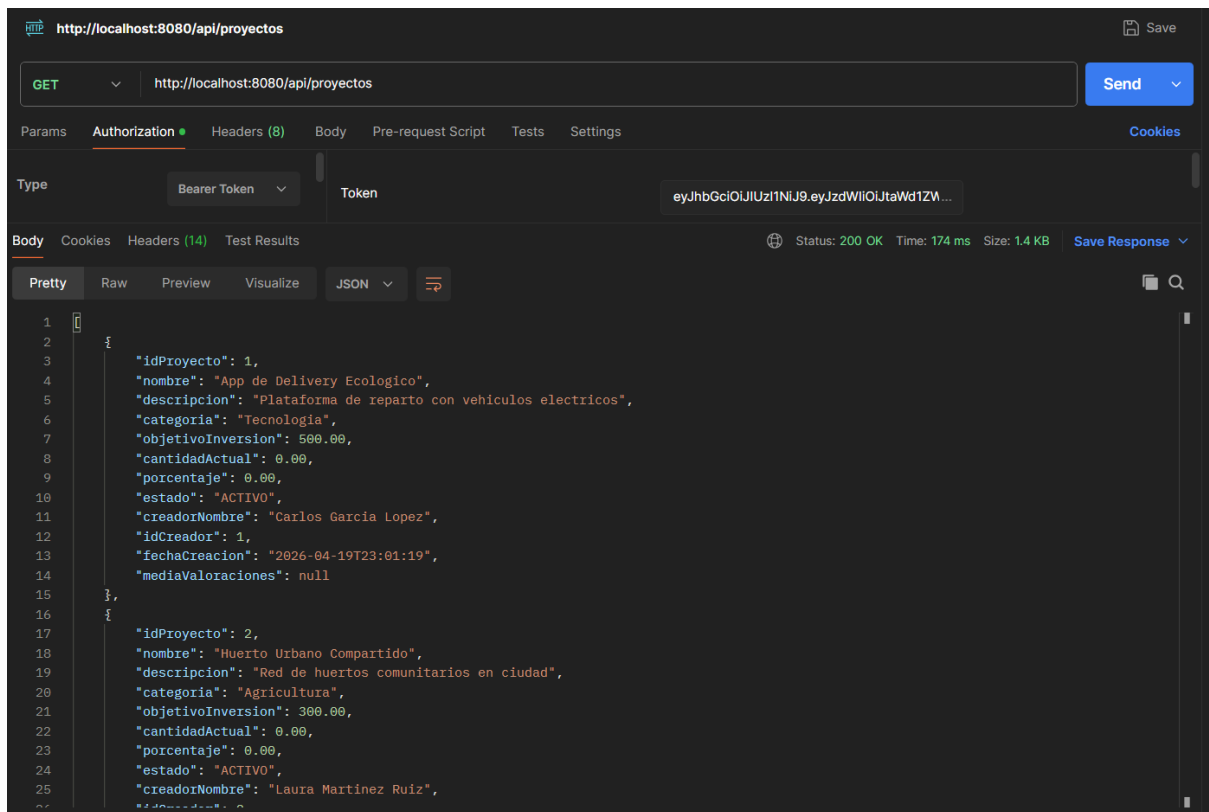


Ilustración 14. Petición GET /api/proyectos con Bearer token y respuesta de la API.

8.5 Repositorio GitHub

El código fuente completo del proyecto, incluyendo el backend Spring Boot, el frontend Android y el script SQL, está disponible en el repositorio GitHub del proyecto.

(Adjuntar enlace del github)

8.6 Iniciativa Emprendedora (IPE)

Análisis DAFO:

	Positivo	Negativo
Interno	Fortalezas: App gratuita, sin riesgo económico real, gamificación motivadora, tecnología moderna (Spring Boot + Android)	Debilidades: Solo disponible en Android, requiere conexión a internet, sin notificaciones push aún
Externo	Oportunidades: Creciente interés en educación financiera entre jóvenes, ausencia de competidores directos con gamificación	Amenazas: Plataformas grandes como eToro que añaden modo educativo, dependencia de la tienda Google Play

Propuesta de mejora derivada del DAFO

Para mitigar las debilidades e amenazas, las mejoras prioritarias serían desarrollar una versión iOS que amplíe el público objetivo, implementar notificaciones push para aumentar la retención, y añadir un modo offline básico que permita consultar el perfil y el historial de conexión.

9. Referencias / Bibliografía

- **Documentación oficial de Spring Boot:** <https://spring.io/projects/spring-boot>
- **Documentación de Spring Security:** <https://spring.io/projects/spring-security>
- **JSON Web Token (JWT):** <https://jwt.io>
- **MySQL Connector/J 9.1.0:** <https://dev.mysql.com/downloads/connector/j/>
- **Retrofit2 para Android:** <https://square.github.io/retrofit/>
- **Material Design para Android:** <https://m3.material.io>
- **Documentación oficial de Android:** <https://developer.android.com>
- **BCrypt Password Encoder:** <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>
- **Postman API Testing:** <https://www.postman.com>
- **IntelliJ IDEA:** <https://www.jetbrains.com/idea/>
- **Android Studio:** <https://developer.android.com/studio>
- **HikariCP Connection Pool:** <https://github.com/brettwooldridge/HikariCP>
- **Hibernate ORM:** <https://hibernate.org/orm/>